groundcover

How Kernel Programming Escaped The Kernel

About me

👋 Hi, I'm Anais Dotis Georgiou

- DevRel @ groundcover
- Passionate about developer education and community
- I also enjoy: Soccer, Yoga, and Art

1. Master eBPF
   (*Extended Berkeley Packet Filter*)
2. Become king in the server

Agenda

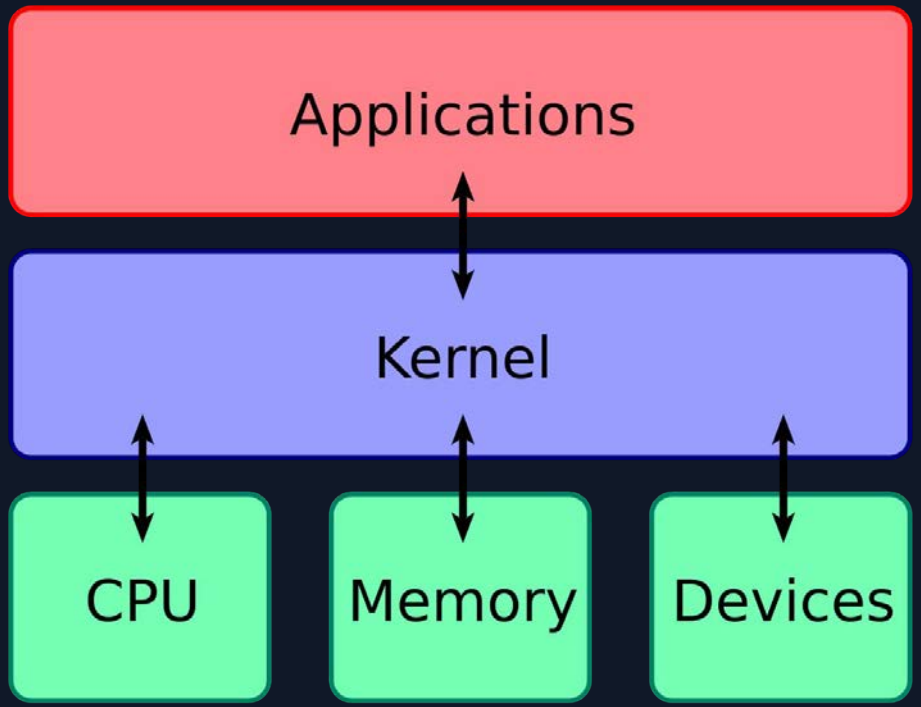
1. What is eBPF and where it came from
2. Use cases
3. How does it work?
4. scripting

groundcover

1

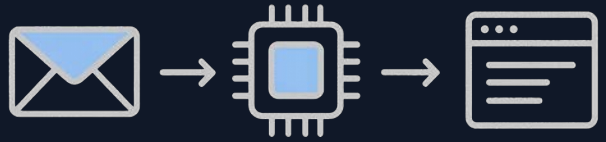
What is eBPF and where it came from

The kernel




```
<body>
  <button id="myButton">Click me!</button>
</body>
```

```
document.getElementById("myButton").addEventListener(
    "click", function() {
        alert("You clicked the button!");
    },
);
```




```
void tcp_recvmsg(struct sock *sk, struct msghdr *msg, size_t len) {
    // Process the incoming TCP message
}
```

```
int kprobe__tcp_recvmsg(struct pt_regs *ctx) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    bpf_trace_printk("%d\n", pid);
    return 0;
}
```

“eBPF does to the kernel what JavaScript does to HTML”

Brendan Gregg

groundcover

How it all began

The year: ± 2011.

The number of cloud programs is growing rapidly.

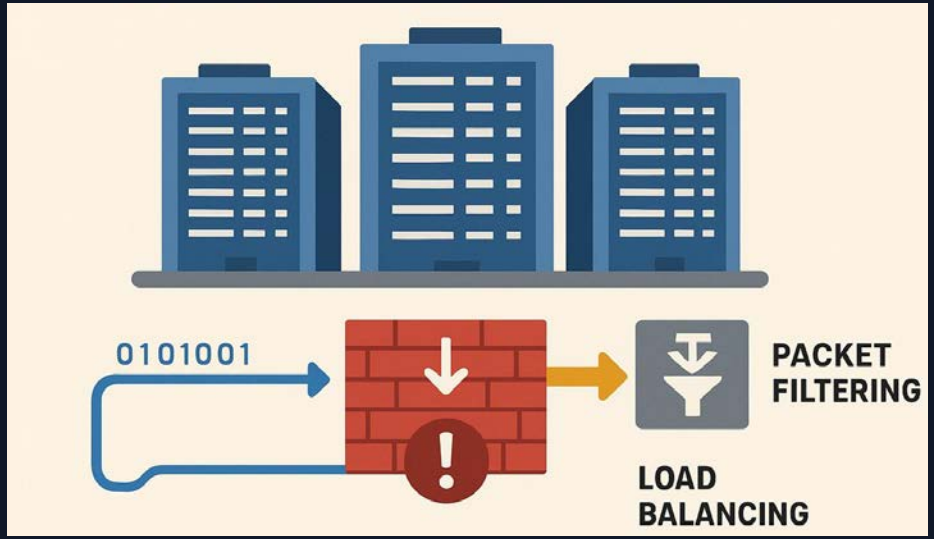
Virtualization is becoming standard.

full story here

How it all began

Large data centers emerge.

Packet filtering and load balancing become the bottleneck.



How it all began

Alexei Starovoitov The father of eBPF

"**Load balancing should run inside the kernel**

*Let's write a new instruction set that the kernel will execute"

How it all began

Daniel is hyped by the idea and joins the effort.

The team realizes the resemblance to (c)BPF.

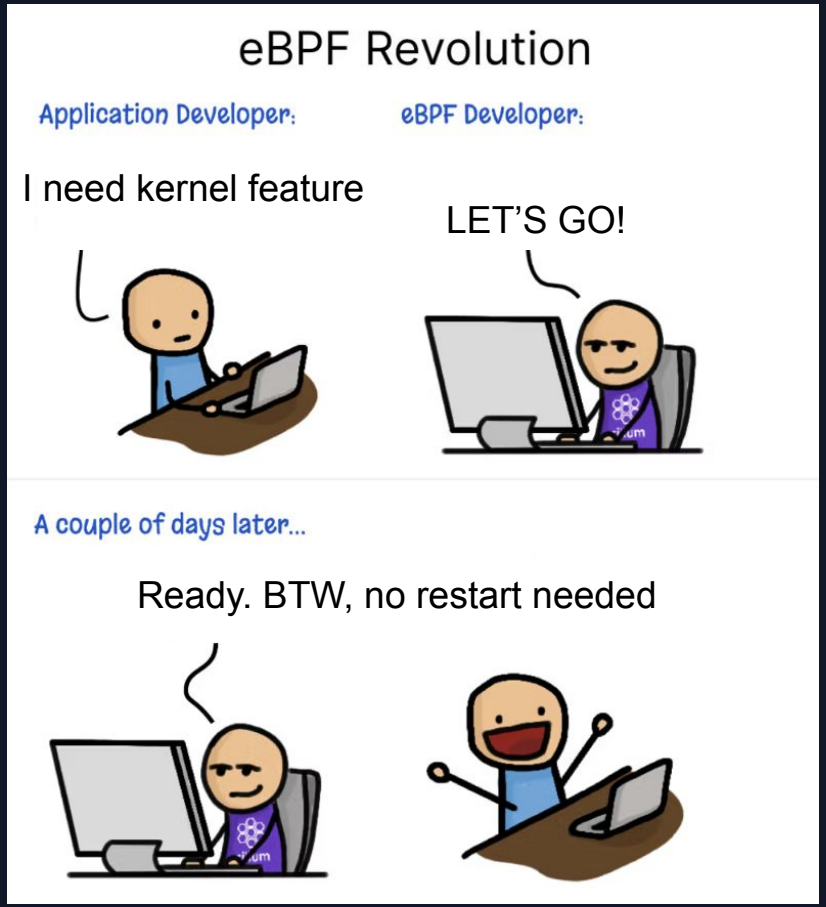
Daniel Borkmann

How it all began

2015 - eBPF proves effective not only for networking. platforms like bcc and bpftrace start showing up.

Brendan Gregg


Application Developer:
I need kernel feature
PLEASE!
Sure, let me ask Linus
1 year later...
All done!
But… I need it in ubuntu!
5 years later...
Ubuntu 30.04 has what u need
what?



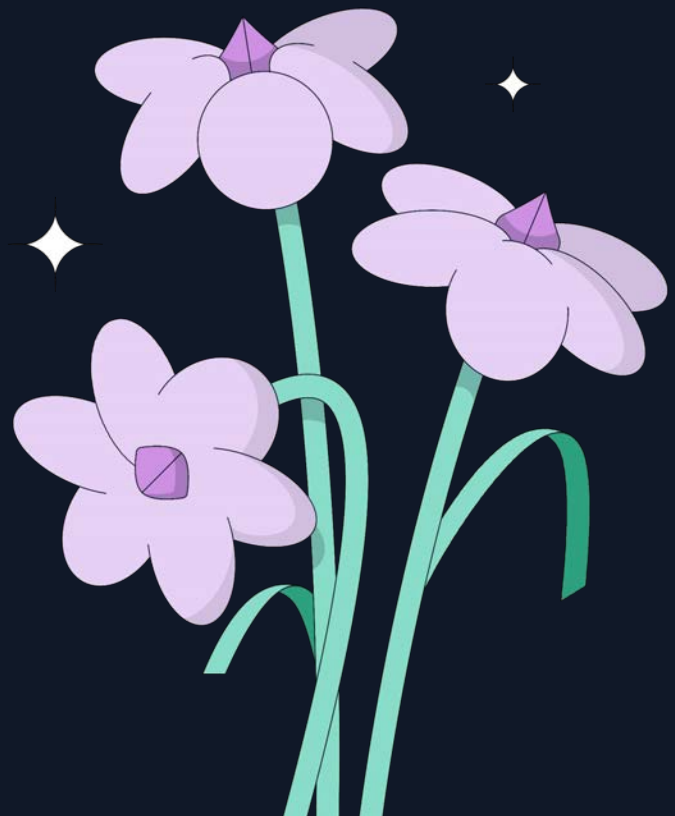
Comic by Philipp Meier and Thomas Graf

groundcover

Use Cases

We will consider eBPF use-cases in 3 realms:

1. Networking
2. Security
3. Observability



eBPF for networking

1. Filter packets
2. Reroute (load balance)
3. Network usage metrics

```
int xdp_process(struct xdp_md *ctx) {
    if (ip->saddr == s_ip) {
        // drop packets from 192.168.1.123
        return XDP_DROP;
    }
    return XDP_PASS;
}
```

eBPF for security

1. File access
2. Process list
3. Firewall

```c
int kprobe__vfs_open(struct file *file)
{
    char filename[256];
    bpf_d_path(
        &file->f_path,
        filename,
        sizeof(filename));
    return 0;
}
```

eBPF for o11y (= Observability)

1. Latency monitoring
2. Traffic visibility
3. Profiling

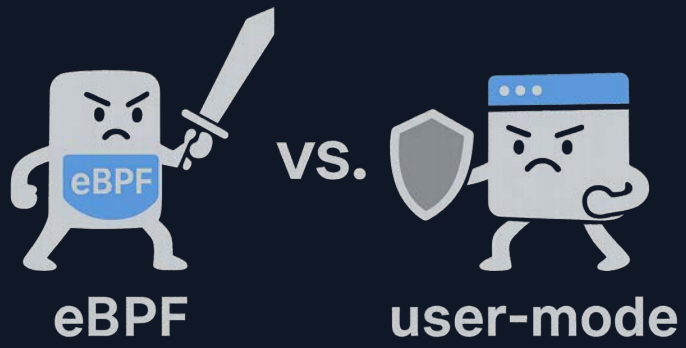
```
void kprobe_sys_write(void* ctx) {
    u64 id = bpf_get_current_pid_tgid();
    u64 timestamp = bpf_ktime_get_ns();
    bpf_map_update_elem(&map, &id, &timestamp, 0);
}
void kretprobe_sys_write(void* ctx) {
    u64 id = bpf_get_current_pid_tgid();
    u64 timestamp = bpf_lookup_elem(&map, &id);
    u64 duration = bpf_ktime_get_ns() - timestamp;
}
```

eBPF for o11y (= Observability)

- ✓ zero code changes
- ✓ no restart
- ✓ full coverage
- ✓ less overhead
- ✓ measure inner-kernel logic

but...

- less context



Use Cases

Security	Networking	Observability
 Tracee	 Cilium	 Prodfiler
 Falco	 Calico	 Pixie
 Tetragon	 Android	 groundcover
	 Katran	 Datadog

How does it work

How does it work

1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

How does it work

1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

```
SEC("kprobe/do_nanosleep")
void BPF_KPROBE(kprobe_do_nanosleep) {
    bpf_printk(
        "process %d sleeping...\n",
        bpf_get_current_pid_tgid() >> 32
    );
}
```

```
$ clang -target bpf
```

How does it work

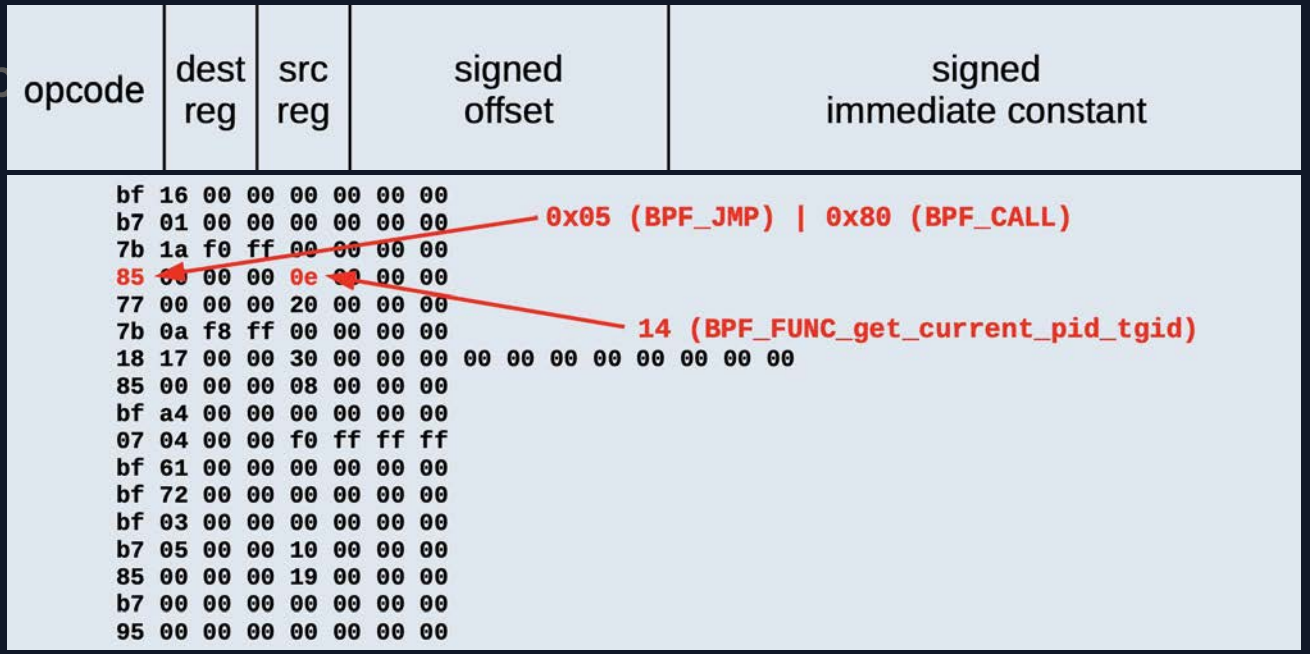
1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

```
#define __BPF_FUNC_MAPPER(FN)           \
        FN(unspec),                     \
        FN(map_lookup_elem),            \
        FN(map_update_elem),            \
        FN(map_delete_elem),            \
        FN(probe_read),                 \
        FN(ktime_get_ns),               \
        FN(trace_printk),               \
        FN(get_prandom_u32),            \
        FN(get_smp_processor_id),       \
        FN(skb_store_bytes),            \
        FN(l3_csum_replace),            \
        FN(l4_csum_replace),            \
        FN(tail_call),                  \
        FN(clone_redirect),             \
        FN(get_current_pid_tgid),       \
        FN(get_current_uid_gid),        \
[...]
```

groundcover

How does it work

1. Write your prob
2. Compilation
3. Verification
4. Attachment
5. Execution



How does it work

1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

The kernel analyzes the code by simulating runs over it.

It enforces a lot of things, including:

- Limited stack size
- Limited instruction count
- No sleeping

How does it work

1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

```
(gdb) disas/r do_nanosleep
Dump of assembler code for function do_nanosleep:
   0xffffffff81b7d810 <+0>:   e8 1b 00 4f ff    callq  0xffffffff8106d830
   0xffffffff81b7d815 <+5>:   55                push   %rbp
   0xffffffff81b7d816 <+6>:   89 f1             mov    %esi,%ecx
   0xffffffff81b7d818 <+8>:   48 89 e5          mov    %rsp,%rbp
   0xffffffff81b7d81b <+11>:  41 55             push   %r13
   0xffffffff81b7d81d <+13>:  41 54             push   %r12
   0xffffffff81b7d81f <+15>:  53                push   %rbx
[...]
```

A trampoline to our JIT-compiled probe is inserted at the start of the function

groundcover

How does it work

1. Write your probe
2. Compilation
3. Verification
4. Attachment
5. Execution

```
static int do_jit(struct bpf_prog *bpf_prog, int *addrs, u8 *image,
                  int oldproglen, struct jit_context *ctx)
{
[…]
        for (i = 1; i <= insn_cnt; i++, insn++) {
[…]
                switch (insn->code) {
[…]
                case BPF_JMP | BPF_CALL:
                        func = (u8 *) __bpf_call_base + imm32;
                        if (!imm32 || emit_call(&prog, func, image + addrs[i - 1]))
                                return -EINVAL;
                        break;
[…]
static int emit_call(u8 **pprog, void *func, void *ip)
{
        return emit_patch(pprog, func, ip, 0xE8);
}
```

E.g., call get_current_pid_tgid

0xe8 is x86 CALL

groundcover

BPF across time

1991 — **cBPF (classic Berkeley Packet Filter)**

- Packet filtering
- Firewalls & packet capture
- *tcpdump*, *wireshark*

2014 — **eBPF (extended Berkeley Packet Filter)**

- Full Virtual-Machine inside the kernel
- Write code (=”probe”) everywhere!
- *tracepoint*, *kprobe*, *uprobe*

2019 — “Unbounded” loops
1M instructions

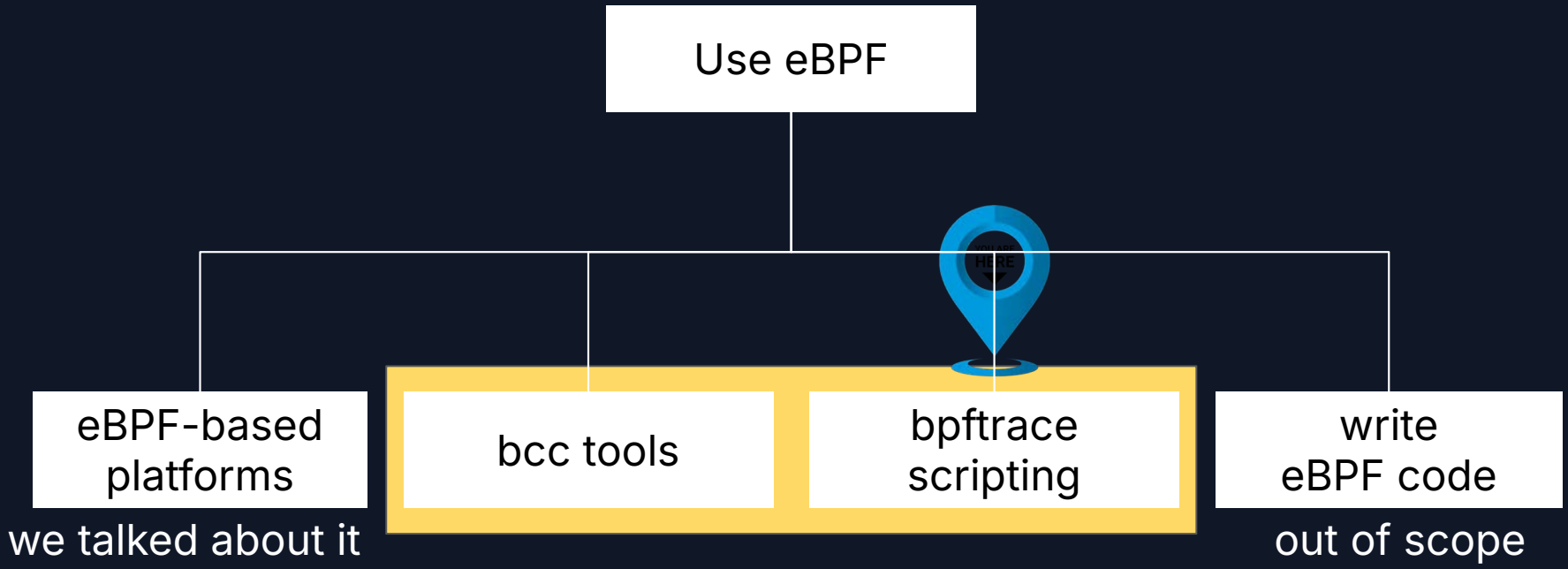
2020 — CO-RE (compile once - run everywhere)

2025 — CPU scheduling

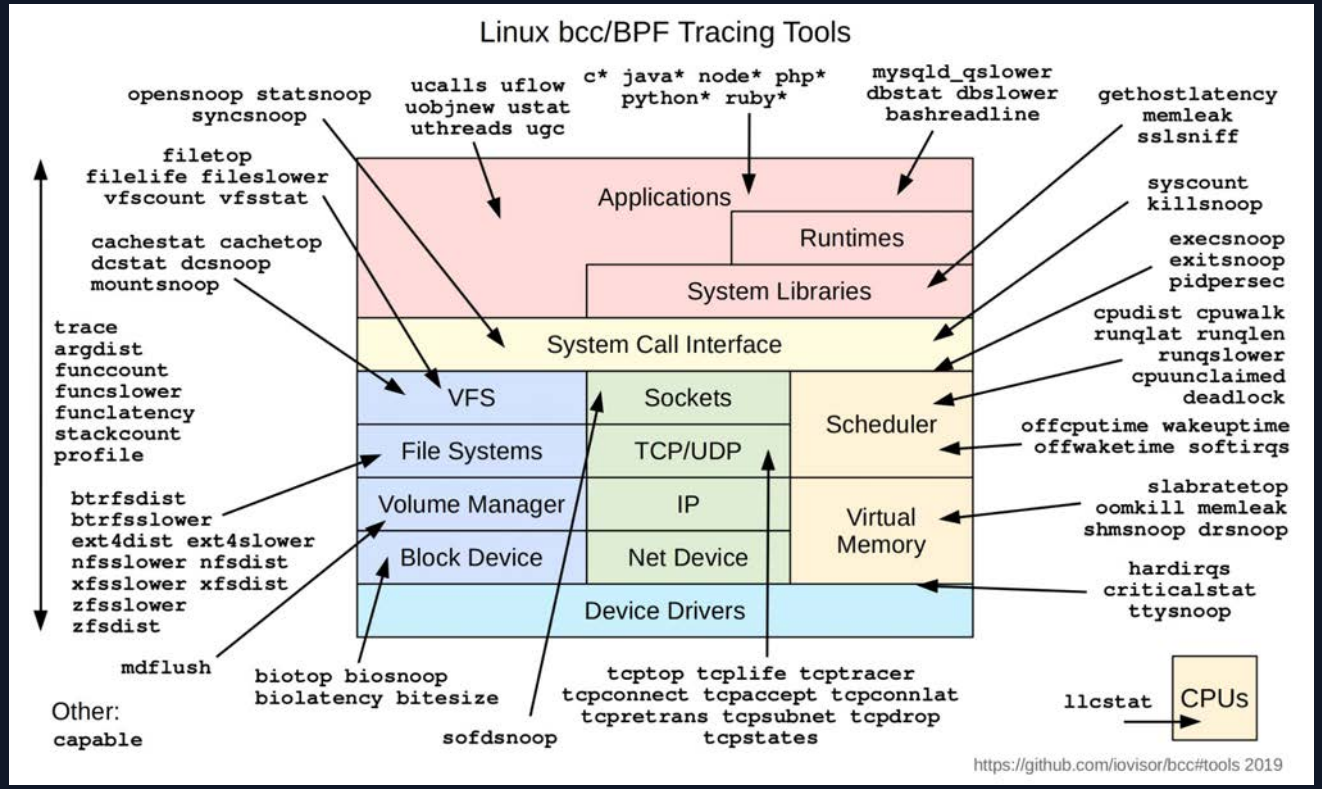
groundcover

4

Scripting


Use eBPF
eBPF-based platforms
bcc tools
bpftrace scripting
write eBPF code
we talked about it
out of scope

bcc tools



groundcover

Demo time!


[I] orishussman@dev-paris ~/p/reversim2025>


[I] orishussman@dev-paris ~/p/reversim2025>


[I] orishussman@dev-paris ~/p/reversim2025>

syscall tracing

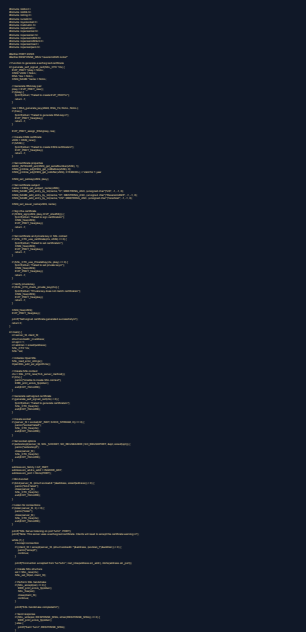
```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_open {
printf("%d opened %s\n", pid, str(args->filename)); }'
```

encrypted traffic debugging

```
.PHONY: server connect

server:
	gcc -o server server.c -lssl -lcrypto
	./server

connect:
	while true; do openssl s_client -connect localhost:29195 -quiet; sleep 1; done
```



```
sudo tcpdump -i any tcp port 29195 -X
```

```
uprobe:/lib/x86_64-linux-gnu/libssl.so.1.1:SSL_write
/comm == "server"/
{
    $len = arg2 > 128 ? 128 : arg2;
    printf("encrypted data from pid %d: %s\n", pid, str(arg1, $len));
}
```

```
sudo bpftrace -e 'uprobe:/lib/x86_64-linux-gnu/libssl.so.1.1:SSL_write
    /comm=="server"/ {printf("%s\n", str(arg1, arg2))}'
```

write to file

```
tracepoint:syscalls:sys_enter_openat
/str(args->filename) == "/tmp/reversim2025"/
{
    @pending_open[pid] = 1;
}

tracepoint:syscalls:sys_exit_openat
/@pending_open[pid]/
{
    printf("PID %d (%s) opened, fd %d\n", pid, comm, args->ret);
    delete(@pending_open[pid]);
    @fds[pid] = (int64)args->ret;
}

tracepoint:syscalls:sys_enter_write
/@fds[pid] == (int64)args->fd/
{
    $len = args->count > 128 ? 128 : args->count;
    printf("PID %d (%s) wrote %d bytes:\n", pid, comm, args->count);
    printf("DATA: %s\n", str(args->buf, $len));
}
```

Thank you!

Ask me anything!

anais.dotis@groundcover.com

github.com/groundcover-com

groundcover